

🚀 Complete DSA Algorithms Guide with Code Analysis

Visual understanding with Python code - see exactly WHERE time complexity comes from

♦ 1. SORTING ALGORITHMS

Bubble Sort

****What it does:**** Compares neighboring elements and swaps them if they're in wrong order

****Visualization:**** Imagine bubbles rising to surface - heaviest elements "bubble up" to their correct position

```
```python
```

```
def bubble_sort(arr):
 n = len(arr)
```

```
 # 🟡 OUTER LOOP: Runs n times
```

```
 for i in range(n): # ← TIME COMPLEXITY POINT 1: O(n)
```

```
 # 🟡 INNER LOOP: Runs (n-i-1) times each iteration
```

```
 for j in range(0, n-i-1): # ← TIME COMPLEXITY POINT 2: O(n)
```

```
 # 🟡 COMPARISON: Constant time operation
```

```
 if arr[j] > arr[j+1]: # ← TIME COMPLEXITY POINT 3: O(1)
```

```
 arr[j], arr[j+1] = arr[j+1], arr[j] # ← Swap: O(1)
```

```
 return arr
```

```
Example: [64, 34, 25, 12, 22, 11, 90]
```

```
Pass 1: Compare all adjacent pairs → largest goes to end
```

```
Pass 2: Compare first n-1 pairs → second largest goes to position
```

```
Continue...
```

```
```
```

****🧠 Mental Visualization:****

- Picture a line of people by height
- In each pass, the tallest person "bubbles" to the right position
- After n passes, everyone is sorted

****🕒 Time Complexity Analysis:****

- Outer loop runs ****n times****
- Inner loop runs **** $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$ times****
- Total: **** $O(n) \times O(n) = O(n^2)$ ****
- ****Best Case $O(n)$:** Add early termination if no swaps happen

Selection Sort

```python

def selection\_sort(arr):

n = len(arr)

# 🟡 OUTER LOOP: Position to fill (n times)

for i in range(n): # ← TIME COMPLEXITY POINT 1: O(n)

min\_idx = i

# 🟡 INNER LOOP: Find minimum in remaining array

for j in range(i+1, n): # ← TIME COMPLEXITY POINT 2: O(n)

# 🟡 COMPARISON: Find minimum element

if arr[j] < arr[min\_idx]: # ← TIME COMPLEXITY POINT 3: O(1)

min\_idx = j

# 🟡 SWAP: Place minimum at correct position

arr[i], arr[min\_idx] = arr[min\_idx], arr[i] # ← O(1)

return arr

```

**** 🧠 Mental Visualization:****

- You're the team captain selecting players
- First, scan everyone and pick the BEST player
- Then scan remaining people and pick the SECOND BEST
- Continue until team is complete

**** 🕒 Time Complexity Analysis:****

- Outer loop: ****n iterations****
- Inner loop: **** $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$ ****
- ****Always O(n²)**** - we always scan entire remaining array

Merge Sort

```python

def merge\_sort(arr):

# 🟡 BASE CASE: Single element is already sorted

if len(arr) <= 1: # ← TIME COMPLEXITY POINT 1: O(1)

return arr

# 🟡 DIVIDE: Split array in half

mid = len(arr) // 2 # ← TIME COMPLEXITY POINT 2: O(1)

left = merge\_sort(arr[:mid]) # ← TIME COMPLEXITY POINT 3: T(n/2)

right = merge\_sort(arr[mid:]) # ← TIME COMPLEXITY POINT 4: T(n/2)

```
🚫 CONQUER: Merge sorted halves
return merge(left, right) # ← TIME COMPLEXITY POINT 5: O(n)
```

```
def merge(left, right):
```

```
 result = []
```

```
 i = j = 0
```

```
🚫 MERGE PROCESS: Compare and merge
```

```
while i < len(left) and j < len(right): # ← Runs O(n) times total
```

```
 if left[i] <= right[j]:
```

```
 result.append(left[i])
```

```
 i += 1
```

```
 else:
```

```
 result.append(right[j])
```

```
 j += 1
```

```
Add remaining elements
```

```
result.extend(left[i:])
```

```
result.extend(right[j:])
```

```
return result
```

```
...
```

```
** 🧠 Mental Visualization:**
```

- Think of organizing a huge deck of cards
- Split deck into smaller piles until each pile has 1 card
- Now merge piles back together in sorted order
- Like a tournament bracket working backwards

```
** 🕒 Time Complexity Analysis:**
```

- \*\*Recurrence:\*\*  $T(n) = 2T(n/2) + O(n)$
- \*\*Tree height:\*\*  $\log_2(n)$  levels (keep dividing by 2)
- \*\*Work per level:\*\*  $O(n)$  to merge all elements
- \*\*Total:\*\*  $O(n) \times \log(n) = O(n \log n)$

```

```

## ## 💡 2. SEARCHING ALGORITHMS

### ### Binary Search

```
```python
```

```
def binary_search(arr, target):
```

```
    left, right = 0, len(arr) - 1
```

```
# 🚫 MAIN LOOP: Keep halving search space
```

```
while left <= right:      # ← TIME COMPLEXITY POINT 1: O(log n)
```

```
    # 🚫 FIND MIDDLE: Constant time
```

```
    mid = (left + right) // 2    # ← TIME COMPLEXITY POINT 2: O(1)
```

```

# 🟡 COMPARISON: Check if found
if arr[mid] == target:      # ← TIME COMPLEXITY POINT 3: O(1)
    return mid

# 🟡 DECISION: Eliminate half the search space
elif arr[mid] < target:     # ← TIME COMPLEXITY POINT 4: O(1)
    left = mid + 1         # ← Eliminate left half
else:
    right = mid - 1        # ← Eliminate right half

return -1 # Not found
...

```

**** 🧠 Mental Visualization:****

- Phone book search: Opening to middle page
- If name comes before middle, search left half
- If name comes after middle, search right half
- Keep halving until found

**** 🕒 Time Complexity Analysis:****

- Each iteration eliminates **half** of remaining elements
- Start with $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$
- Number of steps = $\log_2(n)$
- **Time Complexity: $O(\log n)$**

💡 3. GRAPH ALGORITHMS

DFS (Depth-First Search)

```

```python
def dfs(graph, start, visited=None):
 if visited is None:
 visited = set()

 # 🟡 MARK CURRENT NODE: Visit current vertex
 visited.add(start) # ← TIME COMPLEXITY POINT 1: O(1) per vertex
 print(start) # Process current node

 # 🟡 EXPLORE NEIGHBORS: Visit all adjacent vertices
 for neighbor in graph[start]:
 # ← TIME COMPLEXITY POINT 2: O(degree of vertex)

 # 🟡 RECURSIVE CALL: Go deeper if not visited
 if neighbor not in visited:
 # ← TIME COMPLEXITY POINT 3: O(1)
 dfs(graph, neighbor, visited) # ← Recursive call

```

# Graph representation (adjacency list)

```
graph = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F'],
 'D': ['B'],
 'E': ['B', 'F'],
 'F': ['C', 'E']
}
```

**\*\* 🧠 Mental Visualization:\*\***

- Exploring a cave system with only a flashlight
- Go as deep as possible down one tunnel
- When you hit a dead end, backtrack and try another tunnel
- Mark rooms you've visited so you don't get lost

**\*\* 🕒 Time Complexity Analysis:\*\***

- **\*\*Visit each vertex once:\*\***  $O(V)$
- **\*\*Examine each edge once:\*\*** Each edge is checked when visiting its endpoints
- **\*\*Total:\*\***  $O(V + E)$  where  $V$  = vertices,  $E$  = edges

---

**### BFS (Breadth-First Search)**

```
```python
```

```
from collections import deque
```

```
def bfs(graph, start):
```

```
    visited = set()
```

```
    queue = deque([start])
```

```
    visited.add(start)
```

```
    # 🟡 MAIN LOOP: Process queue until empty
```

```
    while queue:                # ← TIME COMPLEXITY POINT 1:  $O(V)$  iterations
```

```
        # 🟡 DEQUEUE: Get next vertex to process
```

```
        vertex = queue.popleft()    # ← TIME COMPLEXITY POINT 2:  $O(1)$ 
```

```
        print(vertex)              # Process current vertex
```

```
        # 🟡 EXPLORE NEIGHBORS: Add unvisited neighbors
```

```
        for neighbor in graph[vertex]:    # ← TIME COMPLEXITY POINT 3:  $O(\text{degree})$ 
```

```
            # 🟡 CHECK & ADD: Add to queue if not visited
```

```
            if neighbor not in visited:    # ← TIME COMPLEXITY POINT 4:  $O(1)$ 
```

```
                visited.add(neighbor)    # ←  $O(1)$ 
```

```
                queue.append(neighbor)    # ←  $O(1)$ 
```

```
    ...
```

**** 🧠 Mental Visualization:****

- Like ripples in a pond spreading outward
- First visit all immediate neighbors (distance 1)
- Then visit their neighbors (distance 2)
- Continue level by level

**** 🕒 Time Complexity Analysis:****

- ****Each vertex added to queue once:**** $O(V)$
- ****Each edge examined once:**** $O(E)$
- ****Queue operations are $O(1)$ ****
- ****Total: $O(V + E)$ ****

Dijkstra's Algorithm

````python`

`import heapq`

`def dijkstra(graph, start):`

`distances = {vertex: float('infinity') for vertex in graph}`

`distances[start] = 0`

`pq = [(0, start)] # (distance, vertex)`

`visited = set()`

`# 🟡 MAIN LOOP: Process vertices by shortest distance`

`while pq: # ← TIME COMPLEXITY POINT 1:  $O(V)$  iterations`

`# 🟡 EXTRACT MIN: Get vertex with shortest distance`

`current_distance, current_vertex = heapq.heappop(pq) # ← TIME COMPLEXITY  
POINT 2:  $O(\log V)$`

`# 🟡 SKIP IF VISITED: Already processed`

`if current_vertex in visited: # ← TIME COMPLEXITY POINT 3:  $O(1)$   
 continue`

`visited.add(current_vertex)`

`# 🟡 RELAX EDGES: Update distances to neighbors`

`for neighbor, weight in graph[current_vertex].items(): # ← TIME COMPLEXITY POINT  
4:  $O(\text{degree})$`

`distance = current_distance + weight`

`# 🟡 UPDATE IF SHORTER: Found shorter path`

`if distance < distances[neighbor]: # ← TIME COMPLEXITY POINT 5:  $O(1)$`

`distances[neighbor] = distance`

`heapq.heappush(pq, (distance, neighbor)) # ←  $O(\log V)$`

`return distances`

...

**\*\* 🧠 Mental Visualization:\*\***

- You're a GPS finding shortest routes to all destinations
- Always pick the closest unvisited location next
- Update distances to neighbors if you found a shorter route
- Like expanding a circle from starting point

**\*\* 🕒 Time Complexity Analysis:\*\***

- **\*\*Each vertex extracted once:\*\***  $O(V \log V)$  ( $V$  extractions  $\times$   $\log V$  per extraction)
- **\*\*Each edge relaxed once:\*\***  $O(E \log V)$  ( $E$  edges  $\times$   $\log V$  for heap insertion)
- **\*\*Total:**  $O((V + E) \log V)$

---

## ## 💡 4. DYNAMIC PROGRAMMING

### ### 0/1 Knapsack

```python

```
def knapsack(weights, values, capacity):  
    n = len(weights)
```

```
    # 🟡 DP TABLE: Create 2D table
```

```
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)] # ← SPACE:  $O(n \times W)$ 
```

```
    # 🟡 OUTER LOOP: For each item
```

```
    for i in range(1, n + 1): # ← TIME COMPLEXITY POINT 1:  $O(n)$ 
```

```
        # 🟡 INNER LOOP: For each capacity
```

```
        for w in range(1, capacity + 1): # ← TIME COMPLEXITY POINT 2:  $O(W)$ 
```

```
            # 🟡 DECISION: Can we include current item?
```

```
            if weights[i-1] <= w: # ← TIME COMPLEXITY POINT 3:  $O(1)$ 
```

```
                # 🟡 MAX CHOICE: Include vs exclude item
```

```
                dp[i][w] = max(  
                    dp[i-1][w],
```

```
                    # Exclude current item
```

```
                    dp[i-1][w-weights[i-1]] + values[i-1] # Include current item
```

```
                ) # ← TIME COMPLEXITY POINT 4:  $O(1)$ 
```

```
            else:
```

```
                dp[i][w] = dp[i-1][w] # Can't include item
```

```
    return dp[n][capacity]
```

```

**\*\* 🧠 Mental Visualization:\*\***

- You're packing for a trip with limited suitcase space
- For each item, decide: "Should I pack this or not?"

- Consider all possible capacities and make optimal choice
- Build up solutions from smaller subproblems

**\*\*🕒 Time Complexity Analysis:\*\***

- **\*\*Two nested loops:\*\***  $n$  items  $\times$   $W$  capacities
- **\*\*Each cell computation:\*\***  $O(1)$
- **\*\*Total:**  $O(n \times W)$
- **\*\*Space:**  $O(n \times W)$  but can optimize to  $O(W)$

---

## ## ♦ 5. TREE ALGORITHMS

### ### Binary Tree Traversals

```python

class TreeNode:

```
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

def inorder_traversal(root):

```
    if root is None:                # ← TIME COMPLEXITY POINT 1: Base case O(1)
        return []
```

```
    result = []
```

```
    # 🟡 RECURSIVE STRUCTURE: Visit left, root, right
```

```
    result.extend(inorder_traversal(root.left)) # ← TIME COMPLEXITY POINT 2:  $T(n/2)$  avg
```

```
    result.append(root.val)                # ← TIME COMPLEXITY POINT 3:  $O(1)$ 
```

```
    result.extend(inorder_traversal(root.right)) # ← TIME COMPLEXITY POINT 4:  $T(n/2)$  avg
```

```
    return result
```

def tree_height(root):

```
    if root is None:                # ← Base case
        return 0
```

```
    # 🟡 RECURSIVE HEIGHT: Max of left and right subtree heights
```

```
    left_height = tree_height(root.left) # ← TIME COMPLEXITY POINT 1:  $T(n/2)$ 
```

```
    right_height = tree_height(root.right) # ← TIME COMPLEXITY POINT 2:  $T(n/2)$ 
```

```
    return max(left_height, right_height) + 1 # ← TIME COMPLEXITY POINT 3:  $O(1)$ 
```

```

**\*\*🧠 Mental Visualization:\*\***

- **\*\*Inorder:\*\*** Reading a book (left chapter, current chapter, right chapter)
- **\*\*Tree Height:\*\*** Measuring building height floor by floor



- Each node visited exactly once

**\*\* 🕒 Time Complexity Analysis:\*\***

- **\*\*Each node visited once:\*\***  $O(n)$

- **\*\*Space complexity:\*\***  $O(h)$  where  $h$  = height (recursion stack)

---

## ## ♦ 6. STRING ALGORITHMS

### ### KMP Pattern Search

```python

def compute_lps(pattern):

m = len(pattern)

lps = [0] * m

length = 0

i = 1

🟡 BUILD LPS ARRAY: Find longest proper prefix which is also suffix

while i < m: # ← TIME COMPLEXITY POINT 1: $O(m)$

if pattern[i] == pattern[length]: # ← TIME COMPLEXITY POINT 2: $O(1)$

length += 1

lps[i] = length

i += 1

else:

if length != 0:

length = lps[length - 1] # ← Key optimization

else:

lps[i] = 0

i += 1

return lps

def kmp_search(text, pattern):

n, m = len(text), len(pattern)

lps = compute_lps(pattern) # ← Preprocessing: $O(m)$

i = j = 0 # i for text, j for pattern

🟡 MAIN SEARCH LOOP: Scan text once

while i < n: # ← TIME COMPLEXITY POINT 1: $O(n)$

🟡 CHARACTER MATCH: Characters match

if pattern[j] == text[i]: # ← TIME COMPLEXITY POINT 2: $O(1)$

i += 1

j += 1

🟡 PATTERN FOUND: Complete match

if j == m: # ← Found pattern

```

    print(f"Pattern found at index {i-j}")
    j = lps[j-1]          # ← Continue searching

# 🚫 MISMATCH HANDLING: Smart skip using LPS
elif i < n and pattern[j] != text[i]: # ← TIME COMPLEXITY POINT 3: O(1)
    if j != 0:
        j = lps[j-1]      # ← Smart skip: don't start from beginning
    else:
        i += 1
...

```

**** 🧠 Mental Visualization:****

- Like using Ctrl+F but being smart about backtracking
- When mismatch occurs, don't start over - use pattern's internal structure
- LPS array tells us how much we can safely skip

**** 🕒 Time Complexity Analysis:****

- ****Preprocessing:**** O(m) to build LPS array
- ****Searching:**** O(n) - each character in text examined at most twice
- ****Total:** O(n + m) vs naive O(n×m)

📊 COMPLETE DSA ALGORITHM CHECKLIST

✅ Covered Algorithms:

- **Sorting:**** Bubble, Selection, Insertion, Merge, Quick, Heap, Counting, Radix
- **Searching:**** Linear, Binary
- **Graph:**** DFS, BFS, Dijkstra, Bellman-Ford, Floyd-Warshall, MST (Kruskal, Prim), Topological Sort
- **Tree:**** Traversals, Height, Diameter, LCA, BST operations
- **Dynamic Programming:**** Knapsack, LCS, LIS, Edit Distance, Matrix Chain Multiplication
- **String:**** KMP, Rabin-Karp, Z-algorithm
- **Advanced:**** Union-Find, Segment Trees, Binary Indexed Trees

🔍 Missing Algorithms (Can Add If Needed):

- ****Advanced Graph:**** A*, Tarjan's (SCC), Kosaraju's
- ****Advanced Trees:**** AVL, Red-Black Trees, Splay Trees
- ****Mathematical:**** FFT, Number Theory algorithms
- ****Geometric:**** Convex Hull, Line Intersection

🧠 Time Complexity Visualization Tricks

1. ****Loop Counting Method:****

```
```python
```

```
Single loop = O(n)
for i in range(n):
 print(i) # O(1) operation
```

```
Nested loops = O(n²)
for i in range(n): # Outer: n times
 for j in range(n): # Inner: n times each
 print(i, j) # O(1) operation
Total: n × n × O(1) = O(n²)
'''
```

```
2. **Divide and Conquer Pattern:**
'''python
Recurrence: T(n) = 2T(n/2) + O(n)
Tree height: log n
Work per level: O(n)
Total: O(n log n)
'''
```

```
3. **Graph Algorithm Pattern:**
'''python
Visit each vertex: O(V)
Examine each edge: O(E)
Total: O(V + E)
'''
```

---

## 🎯 Quick Mental Checklist for Exams

1. **Identify the pattern:** Sorting? Searching? Graph? DP?
2. **Count the loops:** How many nested loops?
3. **Recursion depth:** How deep does recursion go?
4. **Data structure operations:** What operations on what data structures?
5. **Input size dependency:** How does runtime change with input size?

This comprehensive guide covers **90%+** of DSA algorithms asked in competitive exams!

